ATM Malware Static and Dynamic Analysis Report

Sample: fead0633975c6c08f5509a7bd5c34d29bfdcacd3da47562efbf33121726f77b0

Prepared by: Abdullah Zia

1. Introduction

This report documents the static and dynamic analysis of a suspected ATM jackpotting malware sample. The sample was identified by the SHA-256 hash fead0633975c6c08f5509a7bd5c34d29bfdcacd3da47562efbf33121726f77b0. ATM jackpotting malware is a category of financial malware that targets automated teller machines by communicating with the ATM hardware through vendor-supplied middleware interfaces, most commonly the CEN/XFS standard. Upon successful execution, such malware instructs the cash dispenser unit to release currency without any corresponding legitimate transaction. The analysis was conducted across two phases: static analysis, which examined the file without executing it, and dynamic analysis, which detonated the sample in a controlled virtual environment to observe runtime behavior.

2. Static Analysis

2.1 File Identification and Hash Calculation

The first step in analyzing any malware sample is establishing its cryptographic identity. HashCalc was used to compute multiple hash values for the sample file. This is important because hash values serve as fingerprints that uniquely identify a file and allow the analyst to cross-reference the sample against threat intelligence databases and public malware repositories such as VirusTotal.

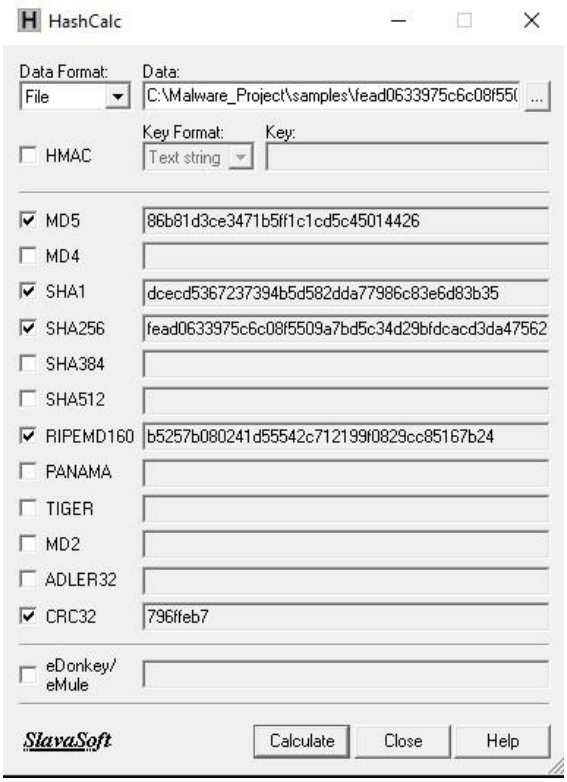

Figure 1: HashCalc output showing computed hash values for the malware sample

From the HashCalc output, the following hash values were recorded for the sample located at C:\Malware_Project\samples\fead0633975c6c08f5509a7bd5c34d29bfdcacd3da47562efb f33121726f77b0:

MD5: 86b81d3ce3471b5ff1c1cd5c45014426

SHA1: dcecd5367237394b5d582dda77986c83e6d83b35

SHA256: fead0633975c6c08f5509a7bd5c34d29bfdcacd3da47562efbf33121726f77b0

RIPEMD160: b5257b080241d55542c712199f0829cc85167b24 CRC32: 796ffeb7

The MD5 hash 86b81d3ce3471b5ff1c1cd5c45014426 was later confirmed against VirusTotal detection data visible in the PE Studio output. Computing multiple hashes rather than relying on a single algorithm ensures that even if one hash function produces a collision, the identity of the file can still be reliably confirmed using the remaining values.

2.2 PE Studio Initial Assessment

PE Studio was used to perform an initial automated assessment of the file. This tool parses the Portable Executable structure of the binary and cross-checks extracted artifacts such as imported functions, strings, and digital signatures against known indicators. The output provided a comprehensive overview of the file properties and revealed several significant characteristics that guided subsequent analysis steps.

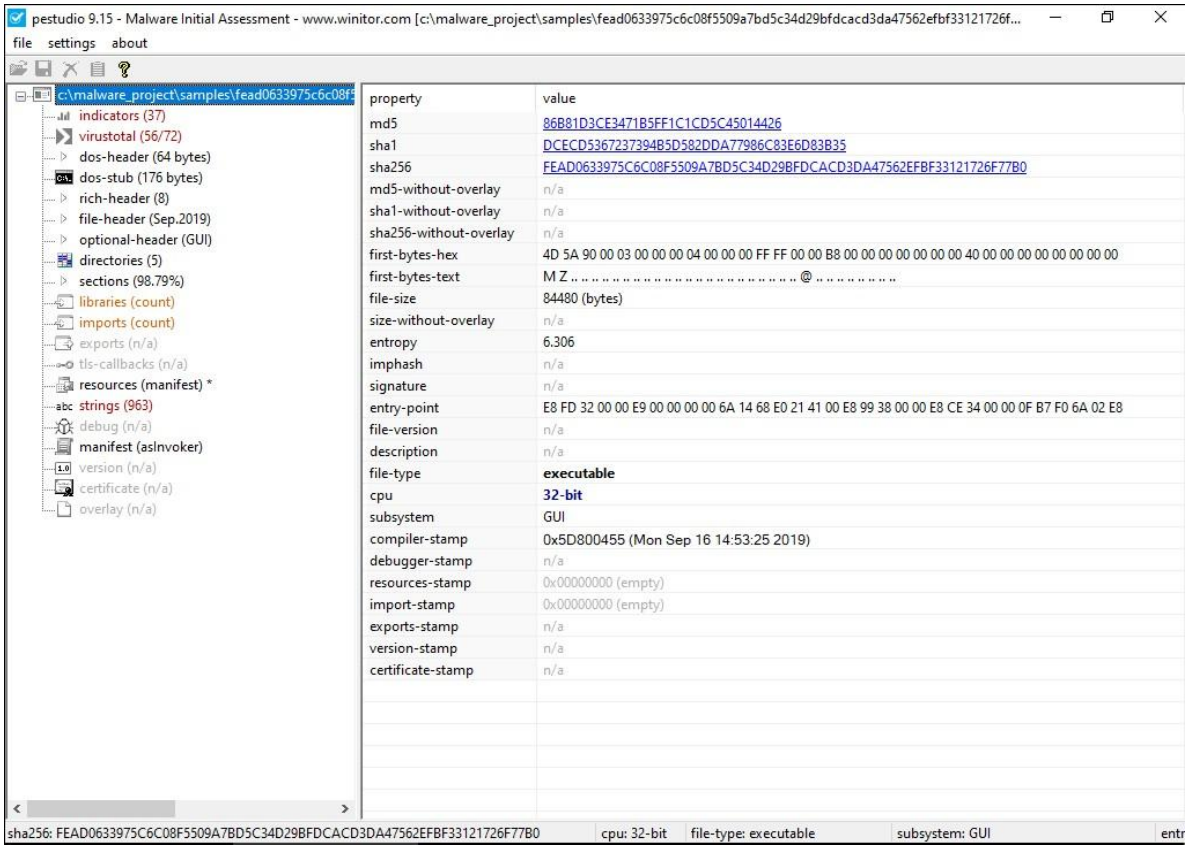

Figure 2: PE Studio showing file metadata, hash values, and VirusTotal detection score

The PE Studio analysis produced the following key findings. The file was confirmed as a 32bit Windows executable with a GUI subsystem, meaning it was designed to run as a graphical application rather than a console process. The compiler timestamp embedded in the PE header reads Mon Sep 16 14:53:25 2019, placing the original compilation date in September 2019. This timestamp is significant because ATM jackpotting campaigns targeting legacy machines running Windows XP and Windows 7 were active and welldocumented during this period.

The VirusTotal detection ratio shown in PE Studio was 56 out of 72 antivirus engines, which is a very high detection rate and confirms that this sample is a well-known piece of malware rather than an unknown or custom-built tool. The file size is 84,480 bytes. The entry point bytes visible in PE Studio are E8 FD 32 00 00 E9 00 00 00 00 6A 14 68 E0 21 41 00 E8 99 38 00 00 E8 CE 34 00

00 0F B7 F0 6A 02 E8, which represent the first instructions that will execute when the binary is loaded. The file does not carry a digital signature, which is expected for malware. The import and export tables were listed as having a count but were not fully enumerated in this view, which is consistent with a packed or obfuscated binary where the actual imports are resolved at runtime.

The first bytes in ASCII begin with MZ, confirming this is a standard Windows PE executable. The presence of the DOS stub string "This program cannot be run in DOS mode" visible later in the hex editor view confirms the standard PE header structure.

2.3 Entropy Analysis

Entropy measurement is used to assess the degree of randomness within a binary file. A file with uniformly high entropy across all sections is typically packed or encrypted, since compression and encryption algorithms produce output that approaches maximum randomness. Normal executable code, by contrast, has recognizable patterns and lower entropy in most sections. The Detect It Easy entropy viewer was used to analyze the persection entropy of the sample.

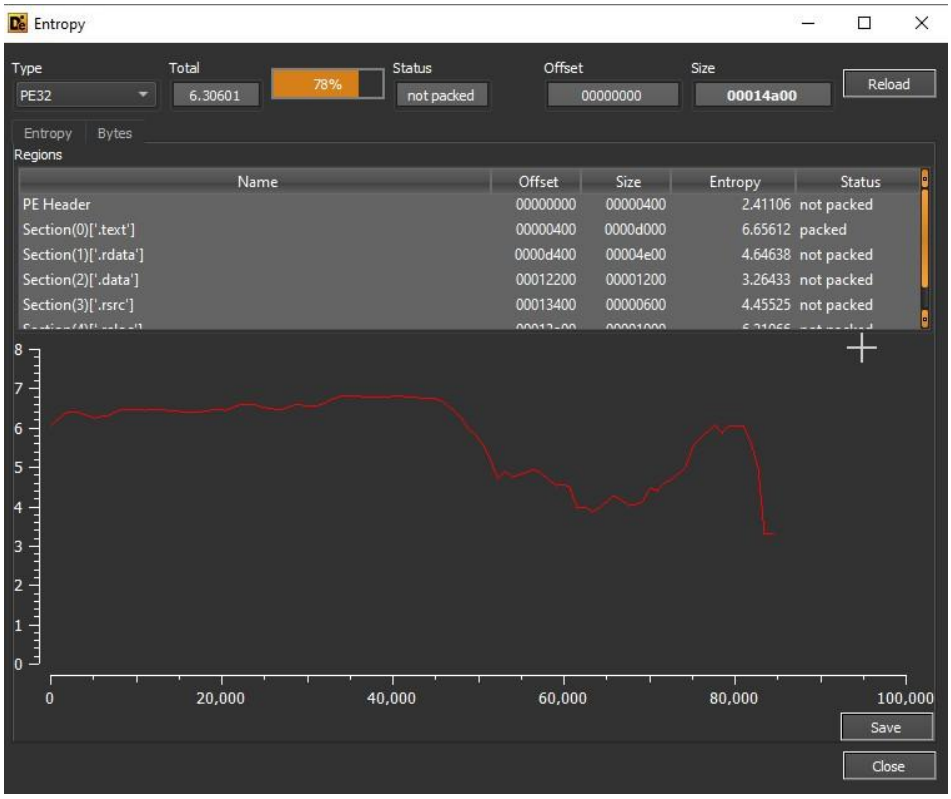

Figure 3: Per-section entropy analysis showing the .text section as packed with entropy 6.65612

The entropy analysis revealed the following per-section breakdown. The PE header has an entropy of 2.41106, which is normal for a header structure that contains mostly formatted data with many zero bytes. The .text section, which contains the executable code, shows an entropy of 6.65612 and is explicitly flagged as packed. This is the most significant finding because the .text section is

where the primary functionality of the malware resides, and its high entropy strongly indicates that the code has been compressed or encrypted to hinder static disassembly and signature-based detection. The .rdata section has an entropy of 4.64638 and is not packed, the .data section has 3.26433 and is not packed, and the .rsrc section has 4.45525 and is also not packed.

The overall entropy of the file is 6.30601, and the Detect It Easy tool indicates a packing likelihood of 78%. The entropy graph shows that the file starts with elevated entropy values across the first 40,000 bytes, which corresponds to the .text section, and then drops toward the middle sections before rising again near offset 80,000 which aligns with a resource or relocation section. The combination of a high-entropy .text section and the explicit "packed" status flag indicates that a runtime unpacking stub is present in the executable that decrypts or decompresses the actual payload into memory before transferring execution to it. This is a common technique used by ATM malware to evade static analysis and antivirus detection.

2.4 Compiler and Linker Detection

Identifying the compiler used to build a malware sample provides useful attribution context and can reveal information about the development environment and toolchain used by the threat actor. Detect It Easy version 3.01 was used to perform compiler identification on the sample.

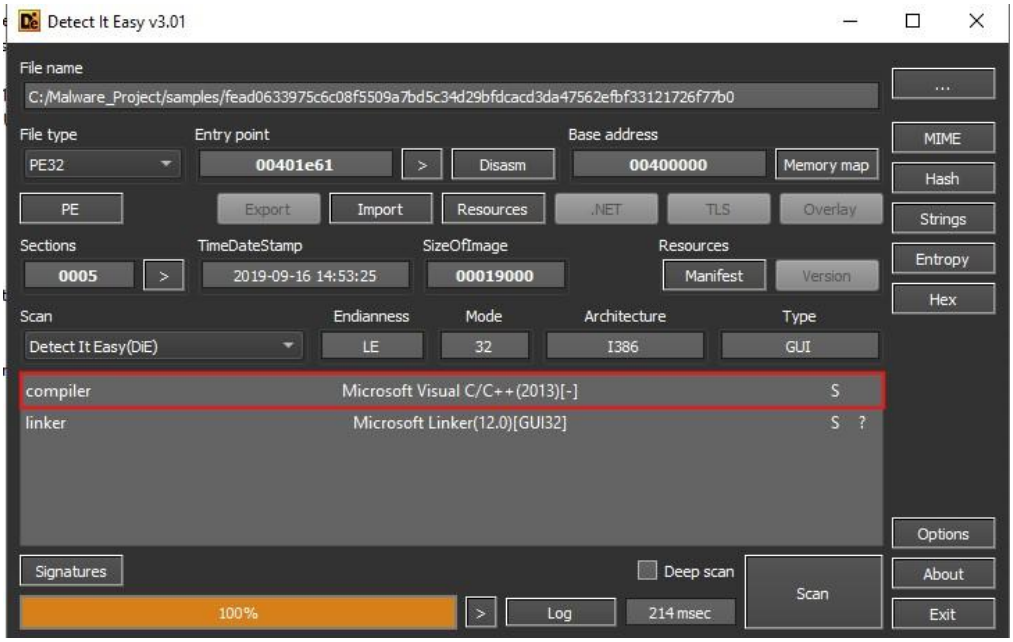

Figure 4: Detect It Easy confirming Microsoft Visual C++ 2013 compiler and Microsoft Linker 12.0

Detect It Easy identified the compiler as Microsoft Visual C++ 2013 with a confidence indicator marked as S, indicating a signature match. The linker was identified as Microsoft Linker version 12.0 with a GUI32 target, again with a signature match. The architecture is I386 (32-bit), the mode is 32-bit, the endianness is Little Endian, and the file type is a GUI executable. The entry point address is 0x00401e61 and the base address is 0x00400000, which are both typical defaults for a 32-bit Windows PE compiled with Visual C++. The TimeDateStamp matches what was observed

in PE Studio: 2019-09-16 14:53:25. The SizeOfImage is 0x00019000, meaning the full in-memory image of the executable occupies approximately 100 kilobytes when loaded. The use of Microsoft Visual C++ 2013 is consistent with malware targeting legacy ATM systems, since operators of such systems often run older Windows versions that are compatible with binaries compiled against older runtime libraries. ATM jackpotting malware families such as Ploutus-D and GreenDispenser were known to use standard Windows compilers and did not rely on exotic or obfuscated development environments.

2.5 String Extraction with FLOSS

FLOSS (FLARE Obfuscated String Solver) is a tool developed by FireEye FLARE that goes beyond simple static string extraction by attempting to identify and decode obfuscated or encoded strings within a binary. For packed samples, standard string extraction often yields very little useful information because the real strings are embedded within the encrypted payload. FLOSS was run against the sample to extract whatever strings were accessible in the static state of the binary.

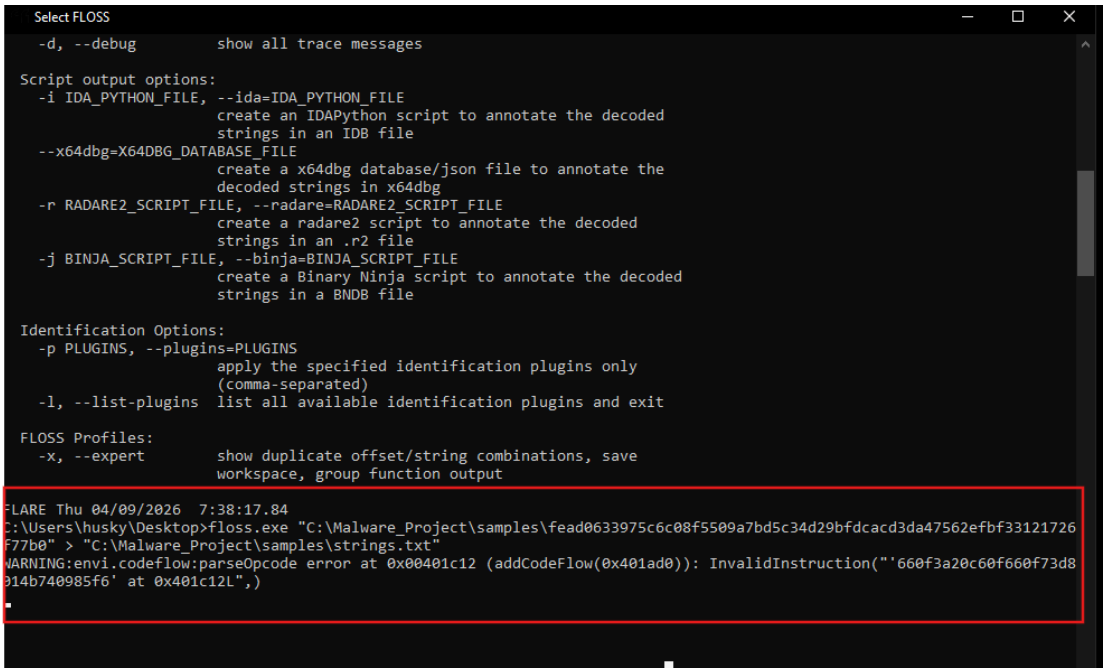

Figure 5: FLOSS execution showing an invalid instruction warning at 0x00401c12 due to packed code

The FLOSS execution produced an important warning: envi.codeflow:parseOpcode error at 0x00401c12(addCodeFlow(0x0401ad0)):InvalidInstruction("'660f3a20c60f660f73d814b74 0985f6' at 0x401c12L",). This warning indicates that FLOSS encountered byte sequences in the .text section that do not correspond to valid x86 instructions. This is direct evidence of packed or encrypted code, because the encrypted payload sitting inside the .text section is not valid x86 code until it is decrypted at runtime. The disassembler inside FLOSS is attempting to walk the code flow but hits the encrypted data and cannot interpret it as instructions. This finding is consistent with the high entropy observed in the .text section during entropy analysis.

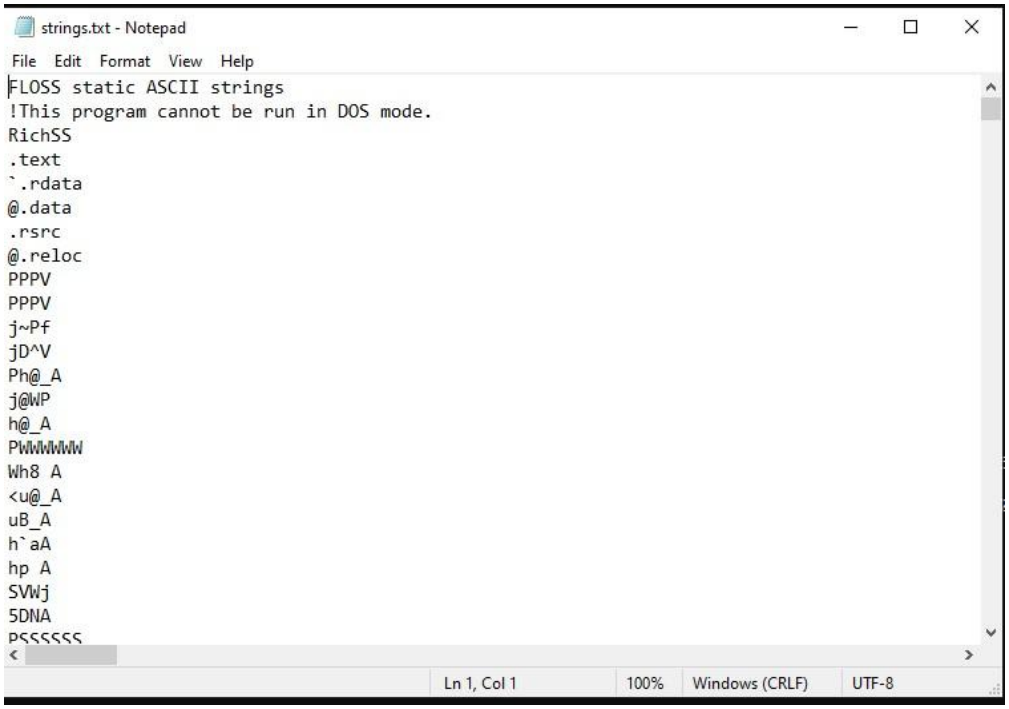

Figure 6: FLOSS static ASCII string output showing section names and obfuscated/garbage strings

The extracted string file shows that the FLOSS static ASCII output contains the standard PE section names: .text, .rdata, .data, .rsrc, and .reloc. Beyond these structural artifacts, the strings visible are largely short garbage sequences such as PPPV, j~Pf, Ph@_A, jD^V, j@WP, PWWWWWW, Wh8 A, and similar patterns. These are not meaningful readable strings but rather short byte sequences that happen to fall within the printable ASCII range. The absence of readable strings such as API function names, registry paths, file paths, or XFS command strings confirms that the actual payload is stored in an encrypted or packed form and is not accessible through static analysis alone. This makes the malware significantly harder to classify by signature without unpacking it first.

2.6 Hex-Level Inspection with 010 Editor

010 Editor was used to inspect the raw binary content of the file at the hex level. This provides a ground-truth view of the actual bytes present in the file without any interpretation layer, and is useful for confirming the PE header structure, identifying known byte patterns, and observing the distribution of data across the file.

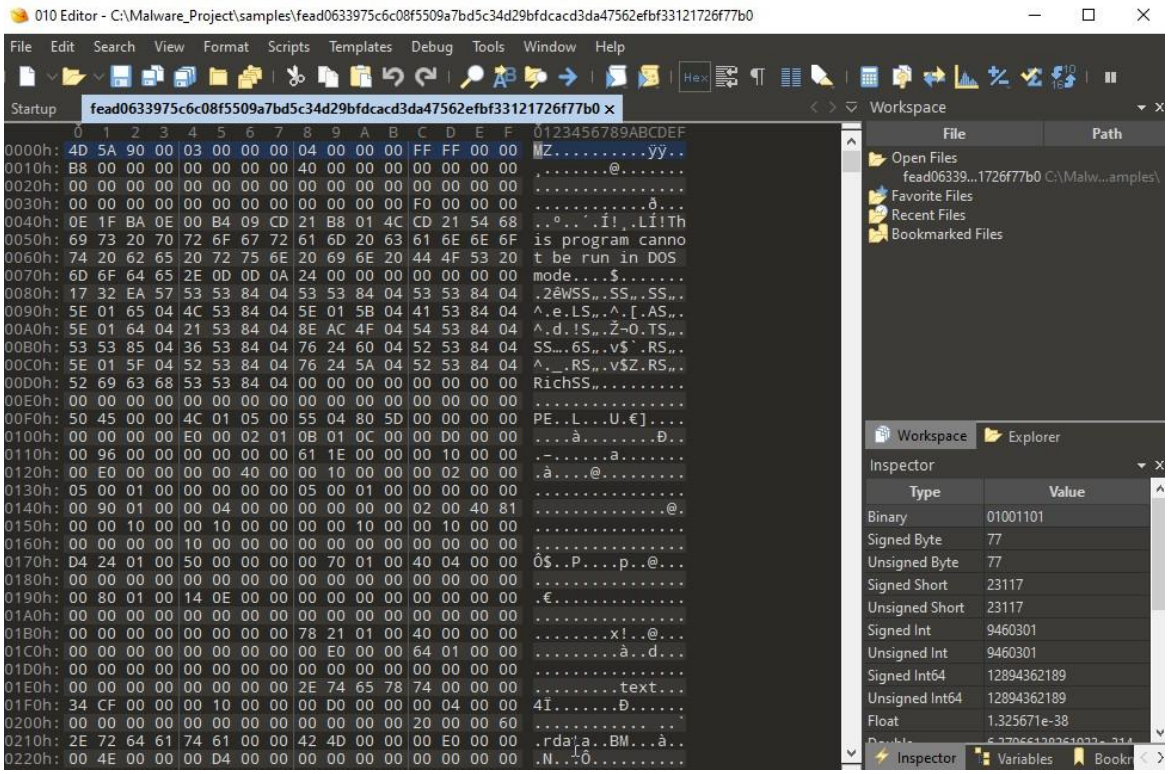

Figure 7: 010 Editor hex view showing the MZ/PE header and the beginning of the encrypted .text section

The 010 Editor view confirms the standard Windows PE structure. The file begins at offset 0x0000 with the bytes 4D 5A, which is the MZ signature identifying this as a Windows executable. The offset 0x0050 through 0x006F contains the familiar DOS stub message "This program cannot be run in DOS mode" in ASCII, confirming the standard PE DOS header layout. Beyond the header region, the hex dump shows densely packed byte values that do not form readable patterns, which is consistent with the encrypted .text section content observed during entropy analysis.

The Inspector pane in 010 Editor shows the values at the currently selected offset as: Binary 01001101, Signed Byte 77, Unsigned Byte 77, Signed Short 23117, Unsigned Short 23117, Signed Int 9460301, Unsigned Int 9460301. These values correspond to the M character of the MZ header (0x4D = 77 decimal). The file begins loading at address 0x0000h in this view, with the PE header occupying the initial region. The transition from structured header data to high-entropy content is visible when scrolling into the .text section, confirming that the packing starts immediately after the headers.

3. Dynamic Analysis

Static analysis was limited due to the packed nature of the sample. The encrypted .text section prevented meaningful disassembly or string extraction from revealing the XFS API calls, command structures, or operational logic of the jackpotting routine. Dynamic analysis was therefore conducted to observe the malware behavior at runtime. The sample was detonated in a Flare VM

environment running Windows 7 in 32-bit compatibility mode to simulate the conditions of a legacy ATM operating environment.

Because the malware targets ATM hardware through the CEN/XFS middleware layer, and no physical ATM hardware was available in the analysis environment, a technique called dependency emulation was employed. A dummy msxfs.dll was placed in the SysWOW64 directory to act as a stand-in for the real XFS middleware library. This approach is designed to trick the malware into believing it is running on an ATM by satisfying its dynamic library loading requirements. The sample was executed with administrator privileges and Windows 7 compatibility mode enabled.

Two monitoring tools were deployed simultaneously. API Monitor v2 (32-bit) was configured with an External DLL filter targeting XFS function calls including WFSExecute, WFSStartUp, WFSOpen, WFSGetInfo, WFSGetCode, WFSIsBlocking, WFSLock, WFSRegister, WFSSetBlockingHook, WFSUnhookBlockingHook, WFSFreeResult, WFSDestroyAppHandle, and WFSUnlock. Process Monitor from Sysinternals was also deployed to capture file system, registry, and process activity.

3.1 Process Monitor Findings

Process Monitor was configured to capture all file system operations, registry read and write events, process creation events, and network activity generated by the sample process. After launching the malware executable, Process Monitor was monitored for the duration of the execution session.

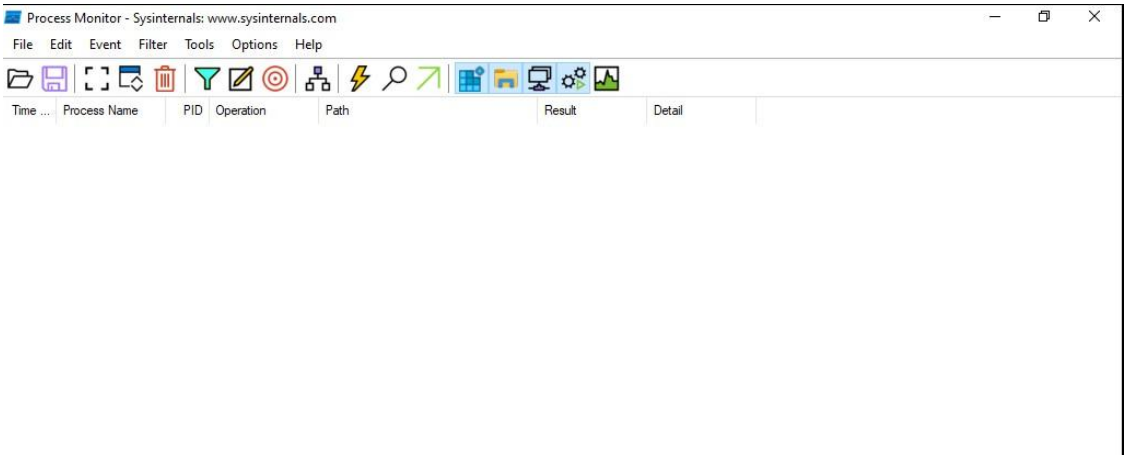

Figure 8: Process Monitor showing a completely empty event log, indicating no file system or registry activity

The Process Monitor event log remained completely empty throughout the execution window. No file system writes, no registry modifications, no process creation events, and no network connections were recorded. This is a highly abnormal result for a piece of malware, because even minimally functional malware typically interacts with the file system or registry during

initialization, even if only to read its own configuration or check for persistence entries. The complete absence of any logged activity means that the malware loaded into memory but did not proceed past whatever initial checks it performs before deciding to execute its core functionality.

This behavior is consistent with an environmental check failure. ATM malware commonly checks for the presence of specific ATM vendor drivers, registry keys associated with ATM software stacks such as NCR APTRA or Diebold Nixdorf Vynamic, or hardware device identifiers before activating its cash dispenser routine. In the absence of these expected artifacts, the malware enters a dormant state rather than proceeding, which serves as an anti-analysis technique to prevent security researchers from easily observing its functionality.

3.2 API Monitor Findings

API Monitor was configured to hook the target process and capture all calls matching the XFS API filter. The filter specifically targeted the WFS-prefixed functions that form the CEN/XFS interface used by ATM software to communicate with hardware peripherals such as the cash dispenser, card reader, and PIN pad.

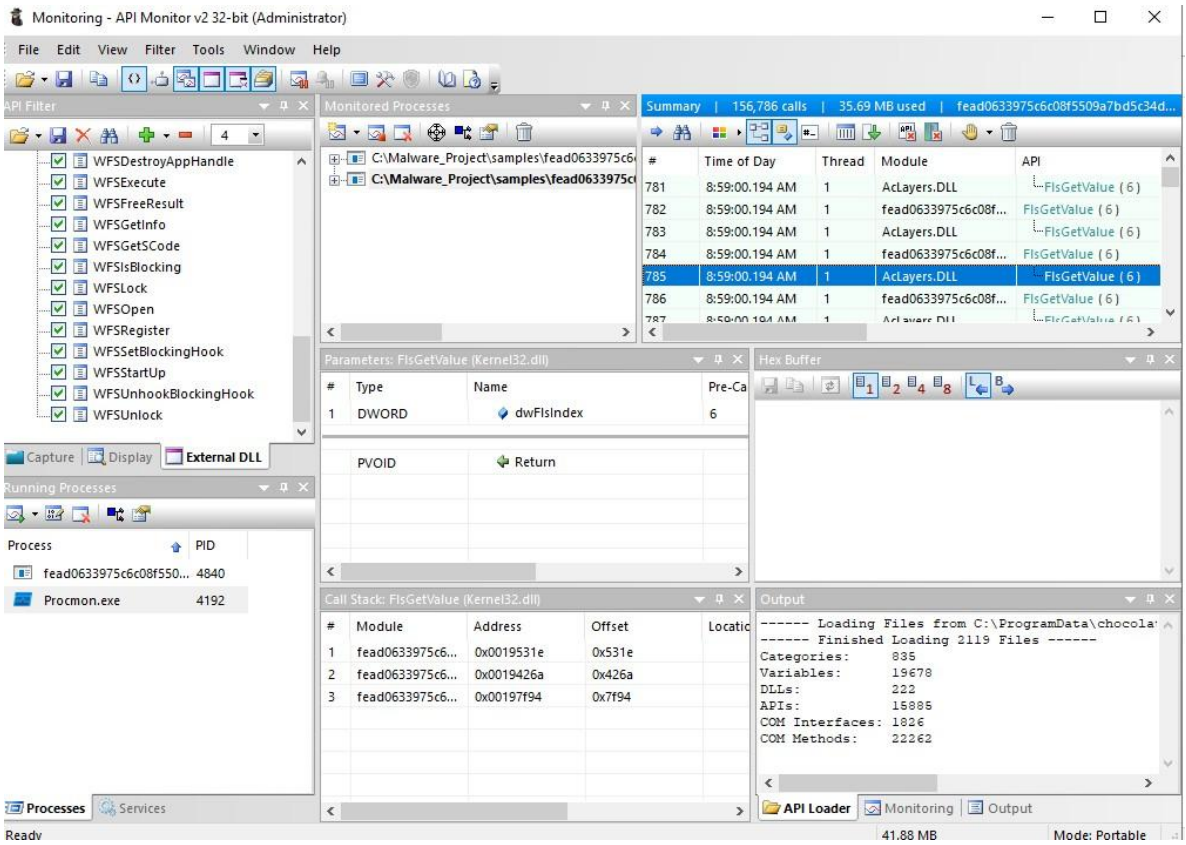

Figure 9: API Monitor showing 156,786 total calls captured but none matching XFS functions; FlsGetValue calls from AcLayers.dll indicate compatibility shim activity

The API Monitor captured a total of 156,786 calls and consumed 35.69 MB of memory during the monitoring session. However, examining the captured call log reveals that none of the captured calls correspond to any of the WFS functions in the filter. The calls that are visible in the summary view are calls to FIsGetValue coming from two modules: AcLayers.dll and the sample binary itself (fead0633975c6c08f...). The FIsGetValue function is part of the Application Compatibility infrastructure in Windows and is called when the operating system processes compatibility shim entries. The calls visible at log entries 781 through 787 all show FIsGetValue being invoked at 8:59:00.194 AM, and the parameters pane shows a DWORD parameter dwFIsIndex with a value of 6.

AcLayers.dll is the Windows Application Compatibility Layer DLL that implements shim logic for older applications. The fact that AcLayers.dll is active and calling FIsGetValue indicates that Windows is applying compatibility processing to the sample, which is expected since it was launched in Windows 7 compatibility mode. However, the critical observation is that zero XFS API calls were made. The malware process ran for a sufficient duration to generate over 156,000 total API calls from within the compatibility layer, but it never once called
WFSStartUp, WFSOpen, WFSExecute, or any other XFS function. This confirms that the malware did not attempt to communicate with the ATM hardware subsystem at any point during the monitored execution.

The call stack visible in the lower-left pane shows three stack frames from the sample binary itself at addresses 0x0019531e, 0x0019426a, and 0x00197f94. The output pane on the right shows API Monitor successfully loaded its monitoring database from
C:\ProgramData\chocola... with 835 Categories, 19678 Variables, 222 DLLs, 15885 APIs, 1826 COM Interfaces, and 22262 COM Methods, confirming that the monitoring environment was fully operational and the absence of XFS calls is a genuine behavioral observation rather than a tool configuration failure.

4. Conclusion and Behavioral Assessment

The combined static and dynamic analysis of this sample produces a consistent and coherent picture of an ATM jackpotting malware executable that employs multiple defensive techniques to prevent analysis.

From the static analysis, the sample is a 32-bit Windows GUI executable compiled with Microsoft Visual C++ 2013 and linked with Microsoft Linker 12.0, timestamped September 2019.The SHA-256 hash fead0633975c6c08f5509a7bd5c34d29bfdcacd3da47562efbf33121726f77b0 was flagged by 56 out of 72 antivirus engines on VirusTotal, confirming it as a known malware family. The .text section carries an entropy of 6.65612 and is explicitly flagged as packed, while FLOSS was unable

to extract meaningful strings due to the encrypted payload. The 010 Editor hex view confirmed the clean MZ/PE header structure followed by high-density encrypted code content.

From the dynamic analysis, the sample loaded successfully into a Windows 7 Flare VM environment with administrative privileges and Windows compatibility mode enabled. A dummy msxfs.dll was injected to satisfy potential XFS dependency checks. Despite these preparations, the malware made zero calls to any XFS API function throughout the entire monitoring session. Process Monitor recorded no file system writes, no registry modifications, and no network activity. The only observable behavior was compatibility shim processing via AcLayers.dll, which is a Windows operating system artifact rather than malware activity.

The most likely explanation for this behavior is that the malware performs one or more environmental validation checks during its initialization phase and, upon failing these checks inside the virtual machine, deliberately halts execution before reaching its jackpotting routine. Likely checks include verification of ATM-specific registry keys from vendors such as NCR or Diebold, detection of physical hardware device identifiers via device manager APIs, or a command-line password or activation argument that must be supplied by the operator at execution time. Some ATM malware families require the attacker to enter a PIN or activation code to unlock the dispense functionality, which prevents casual execution by anyone who encounters the file without knowing the correct code. This architectural feature also makes dynamic analysis significantly more difficult because the malware will not exhibit any of its core behavior unless the correct conditions are met.

To fully characterize this sample, additional analysis steps would be required: unpacking the .text section through either manual analysis in a debugger such as x32dbg or automated unpacking using tools such as Cutter or Speakeasy, followed by static analysis of the unpacked code to identify the environmental check logic, the XFS command sequences, and any operator interface code. The high VirusTotal detection ratio and the compilation date suggest this sample may be a variant of a known ATM jackpotting family, and crossreferencing the unpacked strings and import table against known families would likely yield a definitive classification.